

Optimising Traffic Signal Scheduling: A Game Theory Approach to Reducing Wait Times

Project Team

Syed Subhan Shah Bokhari	22P-9447
Maaz Mustafa	22P-9405
Moiz Ghazanfar	22P-9425

Session 2022-2026

Supervised by

Mr. Usama Musharaf



Department of Computer Science

**National University of Computer and Emerging Sciences
Peshawar, Pakistan**

June, 2026

Student's Declaration

We declare that this project titled “*Optimising Traffic Signal Scheduling: A Game Theory Approach to Reducing Wait Times*”, submitted as requirement for the award of degree of Bachelors in Computer Science, does not contain any material previously submitted for a degree in any university; and that to the best of our knowledge, it does not contain any materials previously published or written by another person except where due reference is made in the text.

We understand that the management of Department of Computer Science, National University of Computer and Emerging Sciences, has a zero tolerance policy towards plagiarism. Therefore, We, as authors of the above-mentioned thesis, solemnly declare that no portion of our thesis has been plagiarized and any material used in the thesis from other sources is properly referenced.

We further understand that if we are found guilty of any form of plagiarism in the thesis work even after graduation, the University reserves the right to revoke our BS degree.

Syed Subhan Shah Bokhari

Signature: _____

Maaz Mustafa

Signature: _____

Moiz Ghazanfar

Signature: _____

Verified by Plagiarism Cell Officer

Dated:

Certificate of Approval



The Department of Computer Science, National University of Computer and Emerging Sciences, accepts this thesis titled *Optimising Traffic Signal Scheduling: A Game Theory Approach to Reducing Wait Times*, submitted by Syed Subhan Shah Bokhari (22P-9447), Maaz Mustafa (22P-9405), and Moiz Ghazanfar (22P-9425), in its current form, and it is satisfying the dissertation requirements for the award of Bachelors Degree in Computer Science.

Supervisor

Mr. Usama Musharaf

Signature: _____

Mr. Riaz Nawab

FYP Coordinator

National University of Computer and Emerging Sciences, Peshawar

Dr. Qasim Jan

HoD of Department of Computer Science

National University of Computer and Emerging Sciences

Acknowledgements

We want to extend my heartfelt thanks to our project supervisor, Mr. Usama Musharaf, for his essential support, understanding, and knowledge during this research. His valuable input and ongoing support played a crucial role in the successful finalization of this report. We are thankful to all the professors for offering the essential resources and a supportive learning atmosphere. Ultimately, we extend our sincere gratitude to all of our colleagues, friends, and family for their steadfast faith, valuable conversations, and encouragement throughout this demanding final year.

Syed Subhan Shah Bokhari

Maaz Mustafa

Moiz Ghazanfar

Abstract

Urban traffic congestion remains a significant challenge due to the inadequacies of fixed-time signal systems that do not respond to varying vehicle demands. This project tackles the issue of optimising traffic signal timing through a Game Theory–inspired approach aimed at reducing vehicle waiting times and promoting an equitable allocation of green time among intersections. The proposed method treats each traffic junction as an independent agent in a non-cooperative game to reach a Nash Equilibrium that strikes a balance between flow efficiency and fairness. The system’s design follows an Agile development methodology, utilizing simulation environments (like Python or SUMO) to evaluate different traffic load conditions. Preliminary results from the simulations suggest a possible decrease in average waiting times and enhanced throughput in comparison to conventional fixed-time control systems. The framework is intended to be scalable for networks with multiple intersections and adaptable to real-time changes in traffic. In summary, this project shows that integrating game-theoretic decision-making models into traffic management systems can produce smarter, more equitable, and efficient urban mobility solutions, paving the way for real-world adaptive traffic control in smart cities.

Contents

1	Introduction	1
1.1	Purpose of the Investigation	1
1.2	Problem Being Investigated	2
1.3	Background and Importance of the Problem	2
1.4	Thesis and General Approach	3
1.5	Criteria for Study's Success	3
1.5.1	Summary	3
2	Review of Literature	5
	Review of Literature	5
3	Project Vision	9
3.1	Problem Statement	9
3.2	Business Opportunity	9
3.3	Objectives	10
3.4	Project Scope	11
3.5	Constraints	11
3.6	Stakeholders Description	12
3.6.1	Stakeholders Summary	12
3.6.2	Key High Level Goals and Problems of Stakeholders	12
4	Software Requirements Specification	15
4.1	List of Features	15
4.2	Functional Requirements	16
4.3	Quality Attributes	16

4.4	Non-Functional Requirements	17
4.5	Use Cases and Use Case Diagram	18
4.6	Architecture Overview	19
4.7	User Interface Screens	20
4.8	Summary	22
5	Iteration Plan	23
6	Iteration Developments (Summary of Core Logic)	25
6.1	Algorithm Design: Baseline and Slipstream (Iteration 1 & 2)	25
6.2	Algorithm Design: Advanced Payoff Matrix (Iteration 3)	25
6.3	Algorithm Design: Predictive Equilibrium (Iteration 4)	26
7	Implementation Details	27
7.1	Traffic Control Interface (TraCI) Integration	27
7.2	Dynamic Metric Extraction	27
7.3	Resolving the Game of Chicken (Nash Equilibrium)	27
8	User Manual	29
8.1	Environment Setup	29
8.2	Compiling the Network	29
8.3	Running the Simulation	29
8.4	Data Extraction	30
9	Conclusions and Future Work	31
9.1	Conclusions	31
10	Iteration 1	35
10.1	Use Case Diagram	35
10.2	Component Diagram	36
10.3	Layer Diagram	37
10.4	Structure Chart	37
10.5	Flow Diagram	38
10.6	Data Flow Diagram (DFD)	38

10.7 Data Dictionary	38
10.8 Activity Diagram	39
10.9 Network Automata / State Machine	40
10.10 Call Graph or Sequence Diagram	40
10.11 Interaction Overview Diagram	41
10.12 Schema Design / ER Diagram	42
10.13 Data Structure Design	42
10.14 Algorithm Design	42
10.15 Development Phase	43
10.15.1 Unit Test	43
10.15.2 Suites or Test Cases	44
10.16 Maintainable Phase	44
10.16.1 CI / CD	44

List of Figures

4.1 Use Case Diagram 19

10.1 Use-Case Diagram 36

10.2 High-Level System Component Diagram. 36

10.3 Smart Traffic Signal Control System Architecture. 37

10.4 Data Flow for the Game-Theoretic Optimization Process. 39

10.5 System Sequence Diagram showing component interactions. 40

List of Tables

2.1 Literature Review 7

Chapter 1

Introduction

This chapter introduces the foundation of the project by explaining the motivation, problem context, and objectives behind this study. It outlines (1) the purpose of the investigation, (2) the problem being investigated, (3) the background and importance of the problem, citing related work, (4) the general approach used in the study, and (5) the criteria for evaluating its success ?.

- The purpose of the investigation
- The problem being investigated
- The background (context and importance) of the problem (citing previous work by others)
- The thesis and general approach
- The criteria for the study's success

1.1 Purpose of the Investigation

The primary purpose of this research is to develop an adaptive traffic signal scheduling system using the principles of **Game Theory**. The goal is to minimise vehicle waiting times and improve traffic flow by modelling intersections as intelligent agents capable

of making real-time scheduling decisions. The project seeks to demonstrate that applying strategic decision-making models, such as the *Nash Equilibrium*, can balance fairness and efficiency in dynamic traffic conditions. This approach contributes to improving urban mobility and reducing congestion through computationally intelligent control systems.

1.2 Problem Being Investigated

Urban traffic congestion is a growing concern in cities worldwide. Traditional traffic light systems operate on fixed-time intervals that fail to account for fluctuating traffic density. This inefficiency leads to excessive vehicle queues, time loss, and environmental impacts due to increased fuel consumption. The specific problem addressed in this project is the **lack of adaptive and fair scheduling mechanisms** that respond effectively to changing traffic demands. Without such adaptive systems, traffic management remains reactive rather than predictive, causing imbalances in flow distribution among intersections.

1.3 Background and Importance of the Problem

Traffic congestion has significant social, economic, and environmental implications. Previous studies, such as those by Abdoos (2020), have proposed multi-agent systems using Game Theory and Reinforcement Learning for signal optimisation. Kolat et al. (2023) explored cooperative learning approaches to improve signal coordination. Although these studies have shown promising results, they often rely on region-specific datasets or lack fairness in scheduling among multiple intersections. This research aims to address these gaps by proposing a model that is both **adaptive** and **equitable**, ensuring smoother traffic flow and reduced waiting times in a scalable framework suitable for developing urban environments.

1.4 Thesis and General Approach

This study proposes a **multi-agent, Game Theory–based framework** in which each traffic intersection acts as a player in a non-cooperative game. Each agent independently adjusts its signal duration based on real-time data and the strategies of neighbouring intersections. The equilibrium state, defined by the *Nash Equilibrium*, represents the optimal configuration of signal timings that balances individual and collective efficiency. The implementation follows an **Agile development methodology**, focusing on iterative simulation, testing, and validation using Python-based modelling tools. The approach emphasises modularity and fairness, ensuring that the scheduling mechanism can adapt to real-world variability.

1.5 Criteria for Study’s Success

The success of this project is determined through a set of measurable criteria:

- Reduction in average vehicle waiting time compared to traditional fixed-time control systems.
- Fair and balanced distribution of green signal durations among intersections.
- Scalability of the model to support multiple interconnected intersections.
- Accuracy and stability of Nash Equilibrium–based optimisation in simulation.
- Positive simulation outcomes demonstrating improved throughput and reduced congestion.

1.5.1 Summary

In summary, this chapter establishes the motivation and foundation for the study. It highlights the need for adaptive, fair, and efficient traffic signal control mechanisms and introduces the use of Game Theory as a solution. The next chapter will review related litera-

ture, summarise previous contributions in this domain, and identify the research gaps the proposed system aims to address.

Chapter 2

Review of Literature

Review of Literature

The literature review for this project examines recent developments in intelligent traffic signal scheduling and optimisation, focusing specifically on **Game Theory**, **multi-agent systems**, and **optimisation-based traffic control models**. Studies conducted from 2020 to 2024 indicate considerable advancements in adaptive traffic management utilising mathematical modelling, artificial intelligence, and decentralised decision-making strategies. Nonetheless, several obstacles remain, including issues with scalability, fairness across intersections, real-time adaptability, and computational constraints in extensive systems.

In [?], H. Li and P. Zhao present a non-cooperative Game Theory framework for traffic signal management, wherein each intersection operates as an independent agent optimising its green light duration according to local traffic circumstances. The research reveals significant improvements in traffic flow efficiency and decreased average waiting times. However, the model's effectiveness deteriorates when addressing multi-intersection coordination, as individual agents tend to focus on local optimisation rather than global outcomes, which can result in unfair signal distribution.

In [?], M. Abdoos and N. Bazzan propose a multi-agent reinforcement learning (MARL) model integrated with Game Theory concepts for dynamic signal control. Each traffic

agent develops optimal strategies through repeated interactions, striving to reach Nash Equilibrium-based coordination. The findings indicate a reduction in congestion and enhancements in throughput relative to fixed-time systems. Nevertheless, this approach necessitates extensive training data and substantial computational power, complicating real-time implementation in large cities with numerous intersections.

In ?, S. Kolat, D. Gora, and M. Nguyen introduce a cooperative Game Theory method for optimising multi-junction traffic systems. The intersections exchange limited information to decrease overall waiting periods collaboratively. While the cooperative strategy promotes smoother traffic flow and fairness, it elevates communication overhead and model complexity. Additionally, it presumes perfect communication among all intersections, which may not be the in real-world scenarios.

In ?, L. Ahmed, K. Farooq, and T. Alvi evaluate hybrid optimisation methods that combine classical algorithms (such as Genetic Algorithms and Particle Swarm Optimisation) with game-theoretic scheduling. Their results suggest hybrid techniques can deliver quicker convergence and robust performance across varying traffic densities. However, they also point out that the need to adjust multiple hyperparameters can make the model susceptible to noise and less responsive to sudden changes in traffic patterns, emphasizing the importance of straightforward, self-adjusting strategies.

In ?, Martinez, F. Singh, and Han assess recent research patterns in intelligent and decentralized transportation systems that incorporate game theory, agent-based modeling, and distributed optimization. The review highlights the primary strength of game-theoretic approaches in harmonizing local decision-making with overarching performance metrics. However, it also highlights challenges in maintaining fairness and computational efficiency, particularly when scaling these methods to city-wide networks with diverse traffic scenarios. The study advocates for lightweight, equilibrium-based models capable of functioning effectively in real time with minimal communication delays.

In summary, these investigations illustrate a clear shift within the domain of adaptive traffic control—from centralized, rule-based systems to intelligent, equilibrium-driven models that facilitate autonomous decision-making. Despite notable advances, there are still persistent research gaps. Importantly, most current systems either compromise fair-

ness for efficiency or demand excessive computational resources. Furthermore, there is a scarcity of frameworks designed for developing cities, where infrastructure and hardware limitations require cost-effective and scalable solutions.

This project seeks to tackle these challenges by creating a **Game Theory–driven adaptive scheduling system** that reduces vehicle waiting times, promotes equitable signal distribution, and maintains computational efficiency. The system will leverage multi-agent coordination and Nash Equilibrium concepts to strike a balance between local optimization and the enhancement of overall traffic flow.

Table 2.1: Literature Review

Sr. No	Year	Paper Title	Methods	Limitations
1	2020	Game-Theoretic Traffic Signal Control Framework	Non-cooperative game model where intersections act as independent agents optimising signal times.	Limited scalability; agents focus on local optimisation rather than network-wide fairness.
2	2021	Multi-Agent Reinforcement Learning for Traffic Signal Optimization	Combined reinforcement learning with Game Theory to coordinate multiple intersections.	High computational cost; requires extensive training data for stable performance.
3	2022	Cooperative Game Theory in Multi-Intersection Signal Control	Applied cooperative Game Theory where agents share traffic data for collective optimisation.	Increased communication overhead; assumes ideal coordination among all intersections.
4	2023	Hybrid Optimisation and Game-Theoretic Scheduling	Integrated classical optimisation algorithms (GA, PSO) with game-theoretic decision models.	Sensitive to hyperparameter tuning; less effective under sudden traffic variations.
5	2024	Intelligent Traffic Management: A Game-Theory Perspective	Surveyed decentralized optimization and multi-agent coordination for smart cities.	Computational intensity; limited focus on fairness and low-cost implementation.

Summary

In summary, the reviewed literature highlights the potential of **Game Theory** and **multi-agent systems** in transforming traffic signal management. Although existing approaches have improved traffic efficiency, they often compromise on fairness or require high com-

putational power. The proposed study builds upon these foundations by introducing a **lightweight, adaptive, and fair scheduling system** based on Nash Equilibrium, capable of achieving real-time optimization without excessive computational requirements.

Chapter 3

Project Vision

3.1 Problem Statement

Urban traffic congestion presents a significant challenge that affects travel efficiency, fuel consumption, and environmental sustainability. Conventional fixed-time traffic signal systems lack adaptability to fluctuating traffic volumes, leading to extended waiting times, increased fuel usage, and reduced mobility. These systems are inherently inflexible and reactive, unable to adjust to real-time changes in road conditions.

The primary problem this project aims to address is the absence of an **adaptive and equitable scheduling model** that can dynamically optimize traffic flow. Current intelligent traffic systems either demand substantial computational resources or rely on localized datasets, making them impractical for large-scale deployment in developing urban areas. This project proposes a **Game Theory–based adaptive scheduling model** that minimizes vehicle waiting time and enhances fairness across intersections through equilibrium-driven decision-making.

3.2 Business Opportunity

As cities evolve toward intelligent infrastructure, there is a growing need for efficient, cost-effective, and scalable traffic management systems. The proposed solution provides

a data-driven alternative for urban planners and smart city developers to improve traffic operations and reduce congestion.

The system has practical applications for:

- **Municipal and traffic management authorities** seeking to upgrade outdated signal systems.
- **Smart city initiatives** implementing adaptive, real-time traffic control frameworks.
- **Research institutions and universities** conducting studies on multi-agent optimization and intelligent transportation systems.

By mitigating congestion and shortening travel delays, the system will reduce emissions, improve commuter satisfaction, and support sustainable urban mobility. Beyond technological innovation, it provides measurable social and environmental benefits that align with the long-term vision of smart city development.

3.3 Objectives

The primary goals of this project are as follows:

- Develop an **adaptive signal scheduling system** grounded in Game Theory principles.
- Model each intersection as an independent agent making decisions based on local and neighboring traffic conditions.
- Apply the concept of **Nash Equilibrium** to achieve fair and efficient optimization across intersections.
- Simulate real-world traffic behavior using Python-based tools to evaluate performance.
- Compare results with traditional fixed-time systems to measure improvements in waiting time, fairness, and throughput.

- Create a scalable, modular framework that can integrate seamlessly with smart city technologies.

3.4 Project Scope

This project focuses on developing a **simulation-based prototype** to evaluate the feasibility of using Game Theory to enhance traffic flow optimization. Each intersection is modeled as an intelligent, self-learning agent capable of dynamically adjusting its signal timing. The simulation will analyze agent interactions and measure the system's ability to reach equilibrium while maintaining overall efficiency.

The project boundaries include:

- **Adaptive signal control** based on traffic density and queue length.
- **Fair scheduling mechanisms** to prevent prolonged delays on specific routes.
- **Simulation and performance analysis** under multiple traffic conditions using synthetic or real datasets.
- **Scalability** to support coordination across multiple intersections with minimal communication overhead.

The initial phase of the project will prioritize simulation and analytical validation rather than physical deployment. The developed model will serve as a foundation for future integration with IoT-based traffic systems or real-time sensor networks.

3.5 Constraints

Several key limitations influence the development and evaluation of the proposed system:

- **Data scarcity:** Availability of region-specific, real-time traffic data is limited, affecting model training and validation accuracy.

- **Computational complexity:** Multi-agent Game Theory simulations require significant processing capacity and optimization for performance.
- **Scalability:** Ensuring effective coordination among intersections without increasing communication latency or computational load.
- **Fairness versus efficiency:** Maintaining a balance between equitable signal distribution and overall system performance.
- **Budget constraints:** Limited resources for acquiring traffic data, computing infrastructure, and advanced simulation tools.

Despite these limitations, the project will focus on developing a lightweight yet effective framework capable of operating within moderate computational and financial constraints.

3.6 Stakeholders Description

3.6.1 Stakeholders Summary

- **Main Stakeholders:** Daily commuters, public transport operators, and road users affected by traffic congestion.
- **Secondary Stakeholders:** Municipal governments, traffic management agencies, and urban policy planners.
- **Development Team:** Software engineers, data analysts, and system architects responsible for model design and implementation.
- **Supervisors and Advisors:** Academic mentors providing technical guidance and project validation.

3.6.2 Key High Level Goals and Problems of Stakeholders

Commuters and Drivers

- **Aspirations:** Reduced waiting times, smoother traffic flow, and lower fuel consumption.
- **Obstacles:** Prolonged delays and inefficiencies caused by static, fixed-timing systems.

Municipal Authorities

- **Aspirations:** Improve traffic efficiency and promote sustainable mobility within budget constraints.
- **Obstacles:** Integration challenges with legacy systems and limited financial resources for modernization.

Development Team

- **Aspirations:** Create a robust, scalable, and accurate Game Theory–based scheduling model.
- **Obstacles:** Computational limits, achieving equilibrium stability, and validation under complex traffic conditions.

Supervisors and Researchers

- **Aspirations:** Advance academic research in adaptive traffic management and Game Theory applications.
- **Obstacles:** Ensuring methodological rigor, reproducibility, and applicability of simulation outcomes in real-world contexts.

Chapter 4

Software Requirements Specification

This chapter outlines the requirements, functionalities, and performance expectations of the proposed **Game Theory–Based Adaptive Traffic Signal Scheduling System**. It defines both functional and non-functional aspects, ensuring that the solution is reliable, scalable, and efficient for practical implementation in real-world intelligent traffic management environments.

4.1 List of Features

The proposed system will include the following major features:

- Adaptive signal scheduling using Game Theory principles for dynamic optimization.
- Multi-agent decision framework where each intersection functions as an independent, intelligent unit.
- Real-time data integration through simulations or IoT-powered traffic sensors.
- Fair distribution of signal timing achieved using Nash Equilibrium modeling.
- Analytical dashboard for live monitoring and visualization of performance metrics.
- Scalable and modular architecture supporting multiple intersections simultaneously.

- Lightweight computational model capable of operating efficiently in low-resource environments.

4.2 Functional Requirements

The following functionalities define the system's operational behavior:

- **Traffic Data Acquisition:** The system collects vehicle flow data such as count, speed, and queue length from sensors or simulated environments.
- **Signal Optimization:** Dynamically computes optimal green and red light durations based on Game Theory algorithms.
- **Agent Interaction:** Facilitates minimal yet effective data sharing between intersections to achieve equilibrium with low communication overhead.
- **Decision-Making Structure:** Applies Nash Equilibrium principles to ensure fair and balanced optimization across all intersections.
- **Performance Assessment:** Evaluates system performance using metrics such as waiting time, fairness index, and throughput.
- **Visualization and Reporting:** Displays real-time data, simulation graphs, and comparative analysis through an interactive dashboard.
- **Parameter Configuration:** Allows traffic engineers to modify parameters such as inflow rates, intersection size, or signal limits.
- **Alerts and Notifications:** Generates alerts when congestion thresholds are exceeded or when equilibrium cannot be achieved.

4.3 Quality Attributes

To ensure reliability and robustness, the following quality attributes will guide development:

- **Reliability:** The system must support stable operation under variable loads and minimize simulation interruptions.
- **Scalability:** Capable of handling multiple intersections without performance degradation.
- **Efficiency:** Optimization cycles should execute rapidly with minimal latency, suitable for real-time simulation.
- **Accuracy:** Signal scheduling results must align closely with theoretical equilibrium outcomes.
- **Usability:** A user-friendly interface will allow configuration and data interpretation with minimal training.
- **Adaptability:** The model should function effectively across varying traffic densities and intersection configurations.
- **Maintainability:** All modules should be well-documented and easily extendable for future system updates or integrations.

4.4 Non-Functional Requirements

The system must fulfill the following non-functional requirements to guarantee performance and maintainability:

- **Performance:** Each optimization cycle should be completed within two seconds to ensure real-time responsiveness.
- **Compatibility:** Must support standard data formats and integrate seamlessly with IoT-based traffic monitoring sensors.
- **Security:** All communication and stored simulation data will be encrypted to maintain confidentiality and integrity.

- **Maintainability:** Codebase must be modular and well-documented for easy debugging and future enhancement.
- **Resource Efficiency:** Operate efficiently on standard computing hardware without requiring specialized processing units.
- **Concurrency:** Capable of supporting multiple simultaneous users and simulation sessions.
- **Availability:** Maintain system uptime of at least 99% during active operation or testing.
- **Data Integrity:** Preserve the accuracy and completeness of stored data including traffic logs and configuration settings.

4.5 Use Cases and Use Case Diagram

The system's use cases describe core user and system interactions:

- **View Traffic Data:** Enables analysts to observe real-time traffic metrics and conditions.
- **Optimize Signal Timing:** Executes Game Theory algorithms to determine optimal signal durations.
- **Run Simulation:** Initiates adaptive scheduling to test performance under simulated traffic conditions.
- **Monitor Performance:** Tracks parameters such as waiting time, fairness index, and throughput.
- **Adjust Parameters:** Allows configuration of vehicle inflow rates, signal timing, and simulation duration.
- **Display Results:** Visualizes traffic performance metrics in graphical format.

- **Generate Reports:** Produces detailed reports comparing adaptive and fixed-time control results.

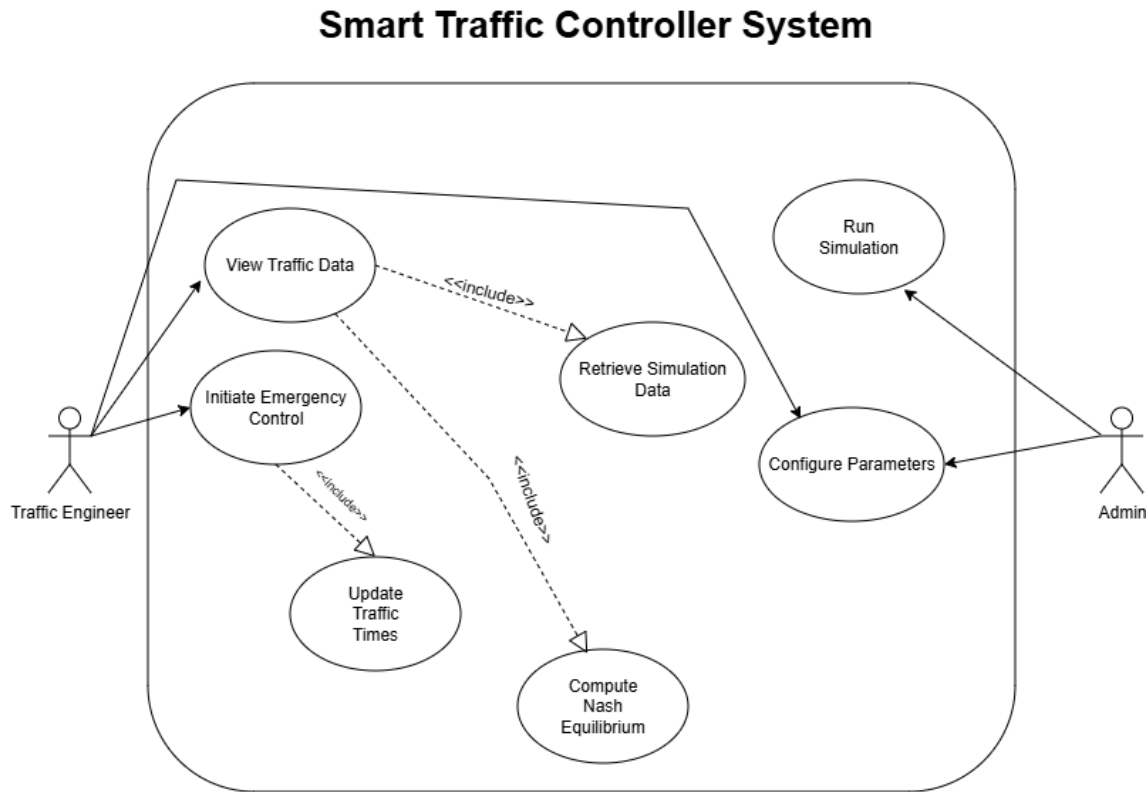


Figure 4.1: Use Case Diagram

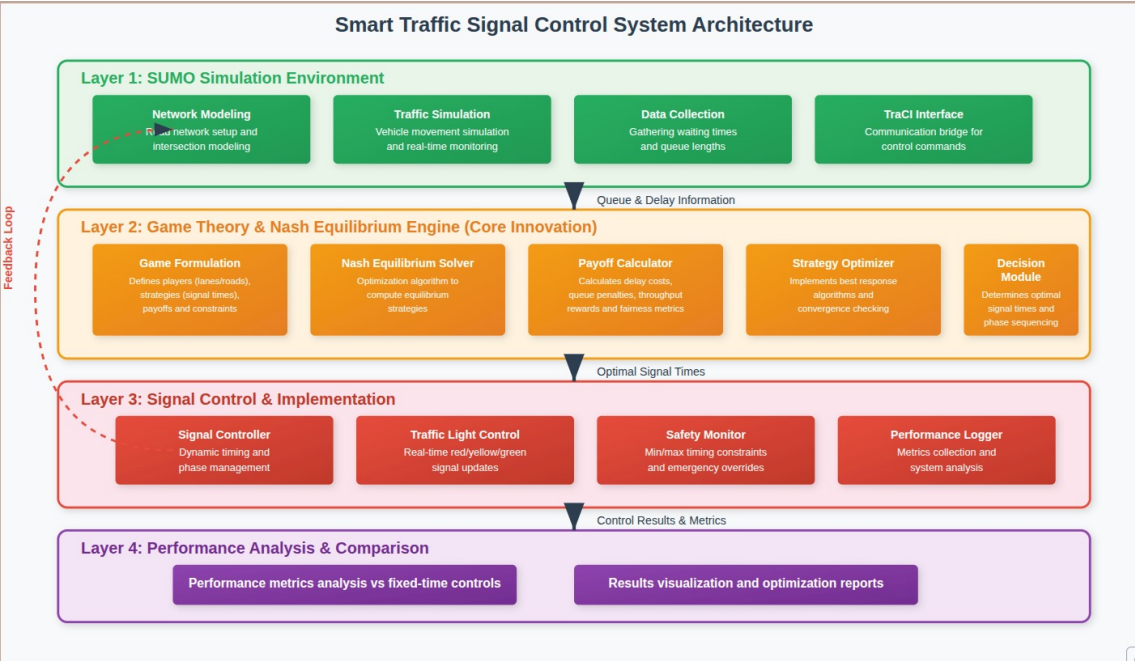
4.6 Architecture Overview

The proposed architecture consists of several interconnected layers that ensure efficient computation, communication, and visualization:

- **Input Layer:** Collects live or simulated data such as vehicle counts and queue lengths from traffic sensors or datasets.
- **Processing Layer:** Implements Game Theory algorithms and handles agent-level computations.
- **Equilibrium Computation Layer:** Calculates Nash Equilibrium for balanced signal timing distribution.

- **Database Layer:** Stores simulation logs, historical data, and performance outcomes.
- **Visualization Layer:** Generates dashboards and performance charts for user interaction.
- **Communication Layer:** Manages data exchange between intersections and central servers with minimal latency.

Figure 4.2: Architecture Diagram



4.7 User Interface Screens

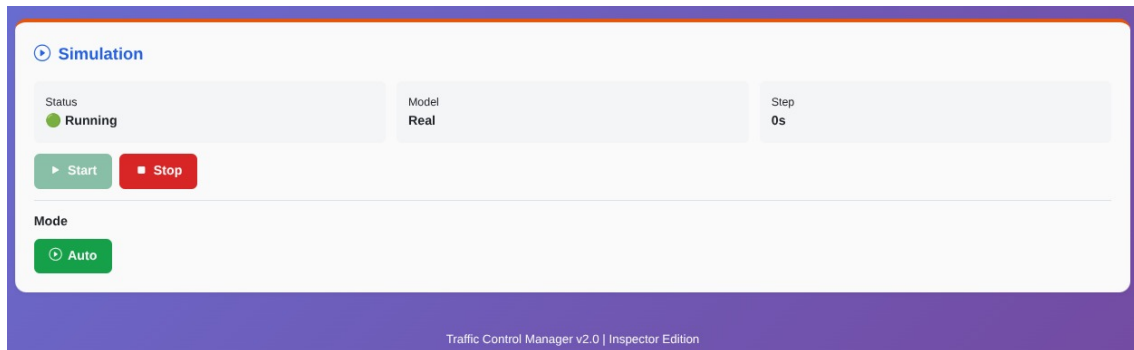
The proposed system will include the following UI components:

1. **Simulation Setup Page:** Configures intersections, traffic volumes, and simulation duration.

Figure 4.3: Simulation Setup Screen

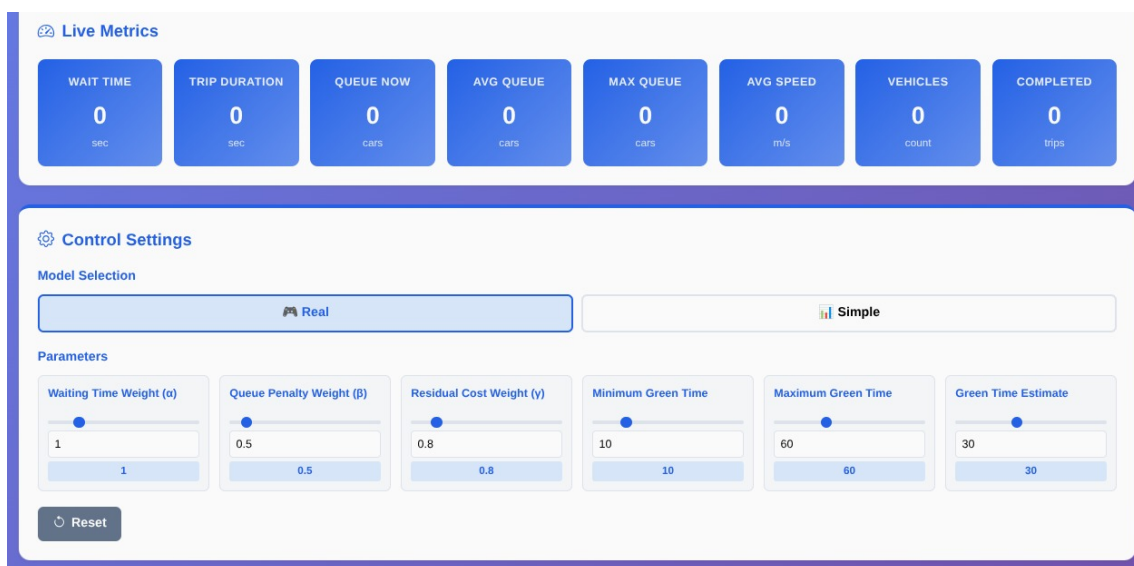
2. **Dashboard:** Displays key metrics such as average waiting time, throughput, and fairness index.

Figure 4.4: Dashboard Interface (Insert Figure Here)



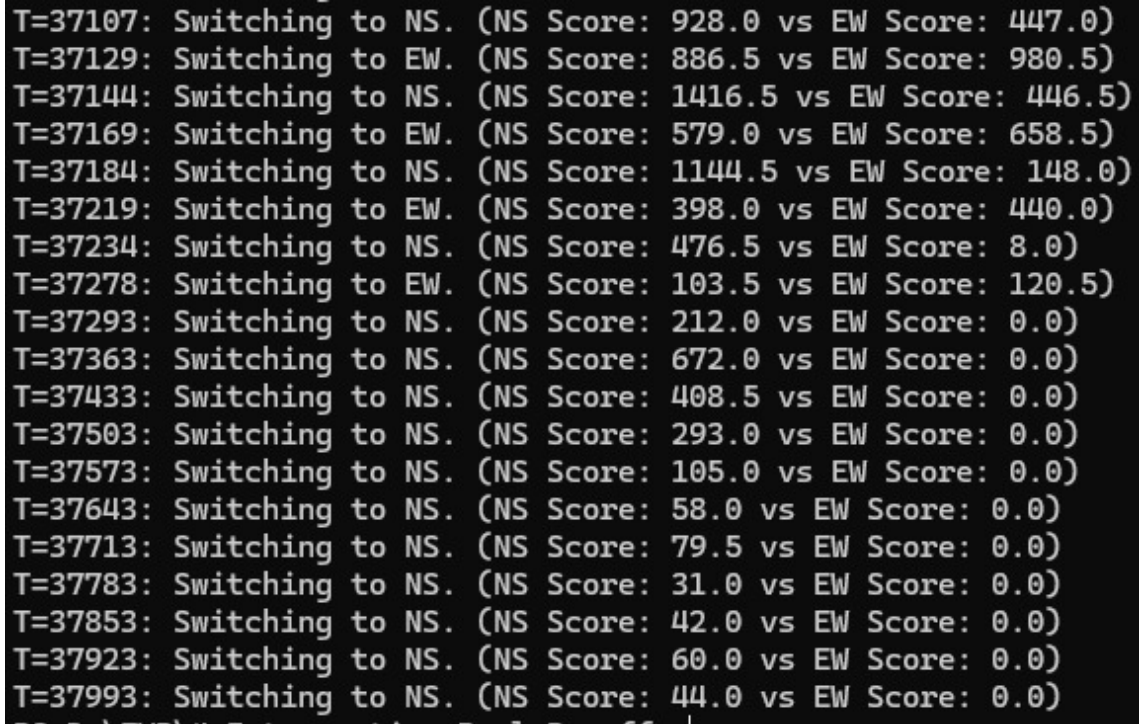
3. **Results Visualization Page:** Provides comparative visual results between fixed-time and adaptive signal scheduling.

Figure 4.5: Results Visualization Screen



4. **Reports and Logs Page:** Shows performance history, simulation results, and data summaries.

5. **Settings and Configuration Page:** Allows adjustment of model parameters, timing constraints, and simulation control preferences.

Figure 4.6: Reports and Logs Screen

```
T=37107: Switching to NS. (NS Score: 928.0 vs EW Score: 447.0)
T=37129: Switching to EW. (NS Score: 886.5 vs EW Score: 980.5)
T=37144: Switching to NS. (NS Score: 1416.5 vs EW Score: 446.5)
T=37169: Switching to EW. (NS Score: 579.0 vs EW Score: 658.5)
T=37184: Switching to NS. (NS Score: 1144.5 vs EW Score: 148.0)
T=37219: Switching to EW. (NS Score: 398.0 vs EW Score: 440.0)
T=37234: Switching to NS. (NS Score: 476.5 vs EW Score: 8.0)
T=37278: Switching to EW. (NS Score: 103.5 vs EW Score: 120.5)
T=37293: Switching to NS. (NS Score: 212.0 vs EW Score: 0.0)
T=37363: Switching to NS. (NS Score: 672.0 vs EW Score: 0.0)
T=37433: Switching to NS. (NS Score: 408.5 vs EW Score: 0.0)
T=37503: Switching to NS. (NS Score: 293.0 vs EW Score: 0.0)
T=37573: Switching to NS. (NS Score: 105.0 vs EW Score: 0.0)
T=37643: Switching to NS. (NS Score: 58.0 vs EW Score: 0.0)
T=37713: Switching to NS. (NS Score: 79.5 vs EW Score: 0.0)
T=37783: Switching to NS. (NS Score: 31.0 vs EW Score: 0.0)
T=37853: Switching to NS. (NS Score: 42.0 vs EW Score: 0.0)
T=37923: Switching to NS. (NS Score: 60.0 vs EW Score: 0.0)
T=37993: Switching to NS. (NS Score: 44.0 vs EW Score: 0.0)
```

4.8 Summary

This chapter described the complete **Software Requirements Specification (SRS)** for the proposed Adaptive Traffic Signal Scheduling System. It outlined functional, non-functional, and architectural requirements ensuring the system's ability to deliver **efficient, fair, and adaptive traffic management** using Game Theory principles.

The proposed system provides a foundation for **intelligent and data-driven urban traffic control**, ensuring scalability, modularity, and future integration with IoT-based smart city infrastructures.

Chapter 5

Iteration Plan

The development of this project was structured into four iterative phases to systematically build, test, and improve the Game Theory-based traffic controller. In this chapter, the plan of execution is discussed in terms of the following phases:

- **Midterm FYP 1 (Iteration 1: Baseline Establishment):** Setting up the SUMO environment, defining the Pakistani mixed-traffic vehicle types, and running a fixed-time 120-second cycle to establish a benchmark for failure rates (teleporting) and wait times.
- **Final FYP 1 (Iteration 2: The "Slipstream" Version):** Designing a custom intersection with dedicated left-turn slip lanes to separate traffic. Implementation of a basic queue-length-based payoff algorithm.
- **Midterm FYP 2 (Iteration 3: Advanced Payoff & Nash Equilibrium):** Transition to a standard 4-lane intersection to stress-test AI. Implementation of the advanced payoff function ($U = \alpha W + \beta D + \gamma Q_{res}$) and discovery of the "Survivor Bias" throughput paradox.
- **Final FYP 2 (Iteration 4: Predictive Queue Equilibrium):** Upgrading the system from a reactive model to a predictive Store-and-Forward model, calculating dynamic arrival rates (q_i) and optimal cycle times (C^*) to clear oncoming traffic.

Chapter 6

Iteration Developments (Summary of Core Logic)

6.1 Algorithm Design: Baseline and Slipstream (Iteration 1 & 2)

The initial algorithm functioned on a reactive basis. Sensors (E2 Detectors spanning 80 meters) monitored queue lengths. The Game Theory controller compared the total queue length of the active Green phase against the waiting Red phase. If the Red phase queue exceeded the Green phase queue by a 10% hysteresis margin, the phase switched. While this improved flow, it introduced a "Starvation Problem" where lanes with few cars were indefinitely ignored in favor of high-volume lanes.

6.2 Algorithm Design: Advanced Payoff Matrix (Iteration 3)

To solve starvation, the algorithm was heavily modified into a 2-Player Non-Cooperative Game (North-South Axis vs. East-West Axis). A weighted utility function was introduced to calculate "Pressure" (Cost of remaining Red):

$$U_i(g) = -(\alpha W_i + \beta D_i + \gamma Q_{residual})$$

Where:

- **W_i (Waiting Cost):** Accumulated waiting time of all vehicles, ensuring long-waiting vehicles eventually force a green light.
- **D_i (Delay Cost):** Webster's Delay formulation, using Q^2 to heavily penalize massive congestion non-linearly.
- **$Q_{residual}$:** The estimated number of unserved vehicles if a standard 30-second green time were applied.

6.3 Algorithm Design: Predictive Equilibrium (Iteration 4)

The final iteration transitioned to a Store-and-Forward mathematical model. Detectors were extended to 150m to track the dynamic arrival rate (q_i). The algorithm computes the exact cycle time (C^*) required to clear the existing queue plus vehicles arriving during the green light:

$$C^*(k) = \frac{\sum_{i=1}^2 \frac{D_i(k)}{S_i^*} + t_L}{1 - \sum_{i=1}^2 \frac{q_i^*(k)}{S_i^*}}$$

This ensures a true Nash Equilibrium where the cycle concludes only when both competing axes reach a queue of zero simultaneously.

Chapter 7

Implementation Details

7.1 Traffic Control Interface (TraCI) Integration

The backbone of the backend logic is the TraCI Python library, which allows real-time manipulation of the SUMO simulation. The script halts the simulation every 1 second to pull metrics from the `laneArea` objects.

7.2 Dynamic Metric Extraction

A critical backend procedure is the extraction of cumulative waiting time. Since SUMO cameras (E2 Detectors) only record the number of vehicles locally, the backend uses `traci.lanearea.getLaneID(det)` to map the camera to its parent lane, and then queries `traci.lane.getWaitingTime(lane_id)` to capture the exact aggregate waiting time (W) of all vehicles on that approach.

7.3 Resolving the Game of Chicken (Nash Equilibrium)

Every evaluation step, the backend constructs a 2x2 payoff matrix representing the "Game of Chicken."

- If both axes demand Green, a catastrophic penalty (-10,000) is applied.
- If both yield, a wasted time penalty is applied to both.

The Social Planner algorithm calculates the Total System Cost for both asymmetric equilibria (NS Go, EW Stop) vs (NS Stop, EW Go). The algorithm strictly enforces the equilibrium that results in the lowest negative utility system-wide, subsequently issuing phase change commands via `traci.trafficlight.setPhase()`.

Chapter 8

User Manual

8.1 Environment Setup

- Ensure Python 3.7+ and Eclipse SUMO (v1.10.0+) are installed.
- Set the SUMO_HOME environment variable to the SUMO installation directory.
- Verify the directory contains the essential XML files: `intersection.net.xml`, `traffic.rou.xml`, `detectors.add.xml`, and `run.sumocfg`.

8.2 Compiling the Network

If modifications are made to the intersection geometry (e.g., changing from slip lanes to a 4-lane model), the network must be recompiled via terminal:

```
netconvert -n intersection.nod.xml -e intersection.edg.xml \  
-x intersection.con.xml -o intersection.net.xml --lefthand=true
```

8.3 Running the Simulation

Do not open SUMO manually. Execute the Python controller via terminal:


```
python controller.py
```

The SUMO GUI will launch automatically. Adjust the delay slider to ~ 100 ms for optimal visual tracking and press the "Play" icon.

8.4 Data Extraction

Post-simulation, run `python convert_queues.py` to flatten the nested `output_queues.xml` into a CSV format suitable for Excel graph generation.

Chapter 9

Conclusions and Future Work

9.1 Conclusions

This project successfully demonstrated that replacing static traffic signal logic with a Game Theory-based adaptive controller significantly optimizes urban traffic flow.

When compared against the Baseline 120s fixed-timer, the Game Theory controller yielded a massive improvement in system throughput. The North (N2C) axis processed 10,200 vehicles compared to the baseline's 6,022 (+69% capacity), and the West (W2C) axis saw a +60% increase. Total network throughput increased from ~32,700 to 40,800 vehicles within the same timeframe, while effectively eliminating "teleporting" (system failure/deadlock) instances from 36 down to 12.

A critical discovery during analysis was the "Survivor Bias" paradox in traffic statistics. Initial `tripinfo` data suggested an increase in average waiting time (from 361s to 500s). However, data analysis revealed that the fixed-time baseline caused gridlocks so severe that thousands of vehicles never completed their trips, omitting their massive wait times from the average. The Game Theory controller cleared these deadlocks, allowing stuck vehicles to finish their routes, which paradoxically raised the average wait time metric while drastically improving actual network travel time and throughput.

Future Work While this study successfully optimized a single, highly congested intersection, future iterations should expand the model to a multi-intersection grid. Integrating

a cooperative multi-agent framework would allow adjacent intersections to share their q_i (arrival rate) data, facilitating a city-wide "Green Wave" synchronization based on networked Nash Equilibria. Additionally, transitioning from simulated E2 cameras to real-world computer vision feeds (e.g., YOLOv8 vehicle detection) would bridge the gap between simulation and physical deployment.

[11pt, a4paper]report

[a4paper, top=2.5cm, bottom=2.5cm, left=2cm, right=2cm]geometry

graphicx

tikz

[colorlinks, linkcolor=blue, urlcolor=blue, citecolor=blue]hyperref

parskip

Chapter 10

Iteration 1

The first iteration is expected to be completed by the midterm of the FYP-1. This chapter will have some of the artifacts based on system design. The requirements analysis section is same for all the systems while the design may vary. There may have two types of designs the structural design or Behavior Design.

10.1 Use Case Diagram

The use case diagram in Figure [10.1](#) illustrates the main actors and their interactions with the Smart Traffic Controller System.

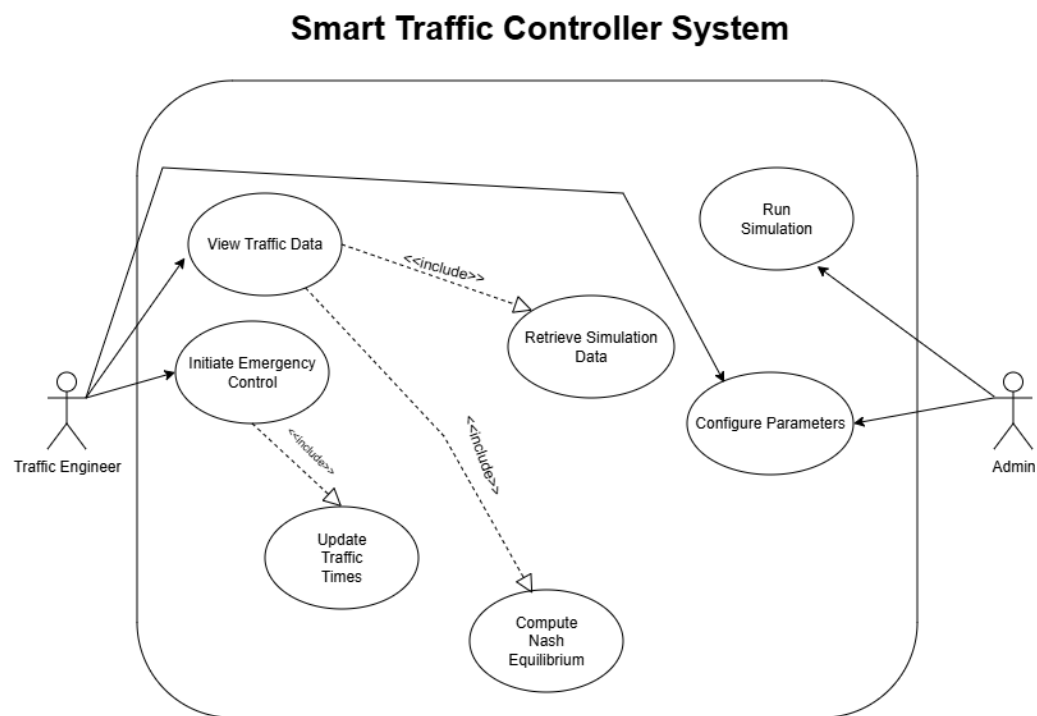


Figure 10.1: Use-Case Diagram

Structural Design

10.2 Component Diagram

The component diagram in Figure 10.2 shows the high-level components of the system and their interactions. It illustrates the data flow between the SUMO simulator, the Traffic Control Interface, and the Application Layer.

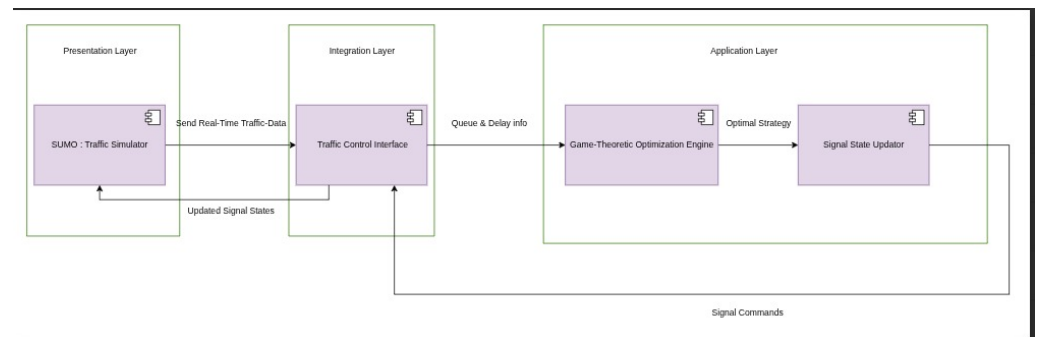


Figure 10.2: High-Level System Component Diagram.

10.3 Layer Diagram

The system architecture is structured in four distinct layers, as shown in Figure 10.3. This layered approach separates concerns, from simulation and data collection (Layer 1) to the core game theory logic (Layer 2), signal implementation (Layer 3), and performance analysis (Layer 4).

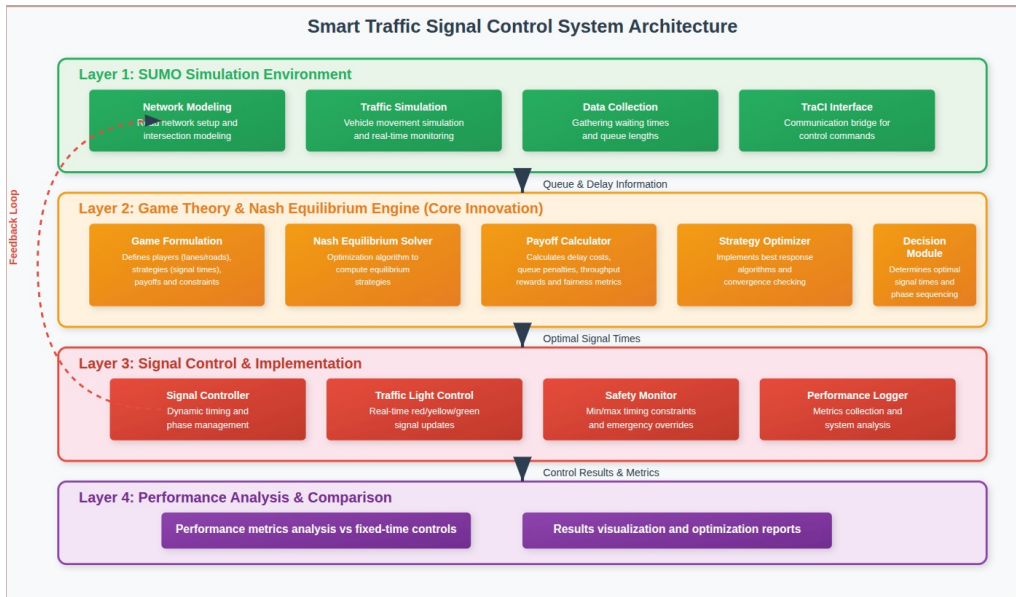


Figure 10.3: Smart Traffic Signal Control System Architecture.

10.4 Structure Chart

The system is divided into four primary modular branches representing the control flow:

- 1) **Simulation Module**: Manages the Eclipse SUMO environment, network compilation, and vehicle route generation.
- 2) **Data Extraction Module**: Utilizes the TraCI API to pull metrics from E2 lane area detectors, mapping lane_id data to aggregate queue lengths and waiting times.
- 3) **Decision Engine**: Houses the core logic, calculating the utility function (U_i) and constructing the 2x2 payoff matrix to resolve the Nash Equilibrium.
- 4) **Execution Module**: Translates the equilibrium strategy into phase-change commands and dispatches them back to the traffic lights via TraCI.

Behavior Design

10.5 Flow Diagram

The primary workflow executes sequentially every second of the simulation:

1. TraCI pauses the simulation step.
2. The Data Extraction module reads dynamic arrival rates (q_i) and cumulative waiting times (W_i).
3. The Decision Engine computes the pressure (cost of remaining Red) for both the North-South and East-West axes.
4. A payoff matrix is constructed representing the "Game of Chicken".
5. The Nash Equilibrium is calculated to determine the optimal asymmetric equilibrium with the lowest system-wide negative utility.
6. If a phase switch is dictated, the execution module triggers a yellow transition phase before switching.
7. The simulation step is advanced.

10.6 Data Flow Diagram (DFD)

Figure 10.4 illustrates the flow of data through the game-theoretic optimization process. It starts with input data from SUMO (Vehicle Counts, Queue Lengths, Waiting Times) and traces how this data is processed to define players, formulate payoffs, solve the Nash Equilibrium, and ultimately compute optimal green times.

10.7 Data Dictionary

The following defines the key variables and structures utilized within the system:

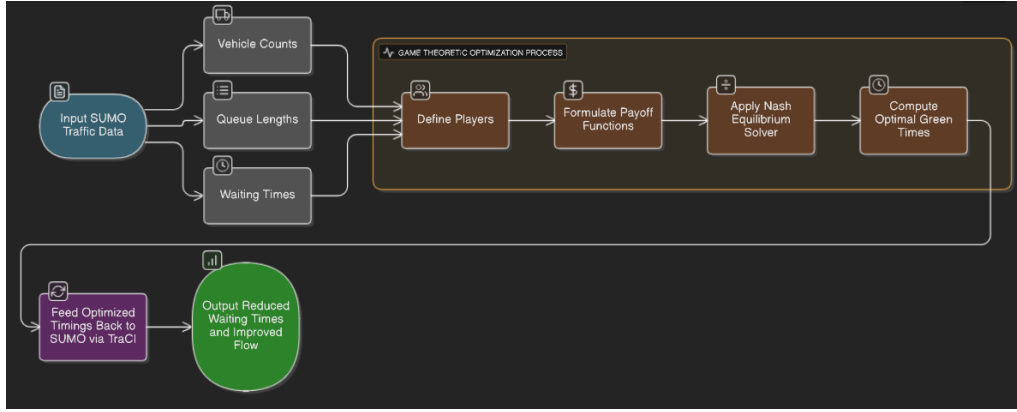


Figure 10.4: Data Flow for the Game-Theoretic Optimization Process.

- **QueueLength (Q):** Integer representing the total number of halted vehicles on a specific approach.
- **WaitingTime (W_i):** Float tracking the cumulative waiting time (in seconds) of all vehicles currently in the queue, extracted via `traci.lane.getWaitingTime()`.
- **PayoffMatrix:** A 2×2 matrix structure containing the calculated utility costs for the (Go, Go), (Go, Stop), (Stop, Go), and (Stop, Stop) action pairs.
- **OptimalStrategy (C^*):** Float representing the dynamically calculated cycle time required to clear the current queue and incoming traffic based on the Store-and-Forward model.

10.8 Activity Diagram

The activity flow begins with system initialization and SUMO network loading. It then enters a parallel state where TraCI simultaneously queries data from the North-South E2 detectors and the East-West E2 detectors. These parallel threads synchronize at a merge node, feeding their respective datasets into the Game Theory evaluation function. Based on the decision branching, the system either maintains the current green signal or triggers a phase transition activity, followed by a time-step advancement activity.

10.9 Network Automata / State Machine

The traffic light controller operates as a finite state machine (FSM) with three primary states: STATE_NS_GREEN, STATE_EW_GREEN, and STATE_YELLOW_TRANSITION. The FSM remains in its current green state until the Game Theory engine determines that the opposing axis has a higher pressure cost. When this condition is met, the FSM moves to STATE_YELLOW_TRANSITION for a fixed duration (e.g., 3 seconds) before securely transitioning to the opposing green state, thereby preventing phase collisions.

10.10 Call Graph or Sequence Diagram

The sequence diagram in Figure 10.5 details the real-time interactions between the system's components. It shows the precise order of messages, from the user initiating the simulation to the Game-Theoretic Engine calculating and returning the optimal signal commands.

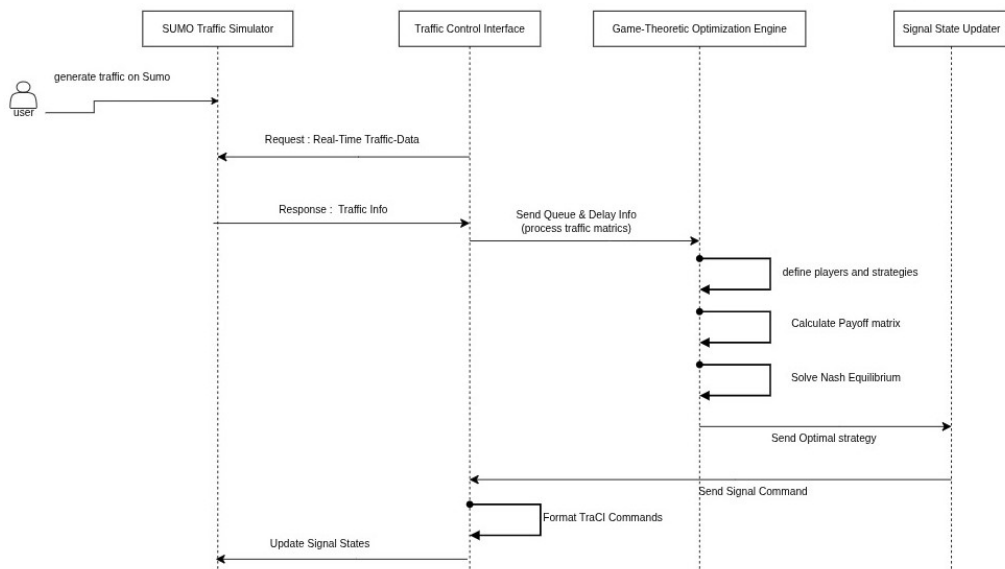


Figure 10.5: System Sequence Diagram showing component interactions.

10.11 Interaction Overview Diagram

The interaction overview combines activity nodes with underlying sequence operations. The main control flow loops through a "Data Collection" macro-activity (involving sequential API calls to `traci.lanearea`), an "Optimization" activity (internal matrix resolution and utility calculation), and an "Actuation" activity (API call to `traci.trafficlight.setPhase`), providing a macroscopic view of the system's operational heartbeat.

For all above designs

10.12 Schema Design / ER Diagram

While the system primarily operates in real-time memory, post-simulation analytics require persistent storage (exported as CSV/XML). The conceptual schema consists of two main entities: **SimulationRun** (RunID [PK], Timestamp, StrategyType, TotalThroughput) and **PhaseMetrics** (StepID [PK], RunID [FK], Phase_State, NS_Queue, EW_Queue, System_Cost). This relationship captures every decision made by the Nash algorithm for later evaluation.

10.13 Data Structure Design

In-memory data management heavily relies on Python dictionaries to map SUMO lane_id strings to their respective real-time metrics. The payoff matrix is implemented as a 2D Python list or NumPy array to facilitate rapid determinant and equilibrium calculations. Communication with TraCI utilizes structured data packets native to the SUMO library, referencing constants such as `traci.constants.VAR_WAITING_TIME`.

System Information

10.14 Algorithm Design

Core Optimization Algorithm: The central algorithm is based on Game Theory.

1. **Data Ingestion:** Receive real-time data (queue lengths, waiting times) for all approaches from the SUMO simulator.
2. **Player & Strategy Definition:** Define each approach (North-South axis vs. East-West axis) as a "player" in a non-cooperative game.

3. **Payoff Formulation:** Calculate utility as $U_i(g) = -(\alpha W_i + \beta D_i + \gamma Q_{res})$. This penalizes high waiting times and massive congestion.
4. **Nash Equilibrium Calculation:** The system solves for the Nash Equilibrium to find a stable state where no single approach can unilaterally change its strategy without causing catastrophic system-wide penalties.
5. **Signal Command Generation:** The optimal strategy is formatted into phase commands and sent back to SUMO via the TraCI interface.

10.15 Development Phase

The project adheres to strict software engineering standards to ensure maintainability and scalability.

- **Comments:** Code is heavily documented, explicitly explaining the mathematical logic behind the C^* cycle time formula and the payoff weighting.
- **Naming Conventions:** Python's PEP 8 standard is strictly followed for all classes, functions, and global variables.
- **Static Analysis of Code:** Flake8 and Pylint are utilized to ensure code quality, catch undefined variables, and enforce formatting rules prior to testing.

10.15.1 Unit Test

Unit tests will be developed using the PyTest framework. Core mathematical modules, such as the utility calculation function, will be tested by passing dummy queue and delay variables and asserting that the resulting pressure cost matches the manually calculated expectation. The matrix solver will be tested against known game-theory scenarios to ensure it correctly identifies the asymmetric equilibrium.

10.15.2 Suites or Test Cases

Testing suites will cover varying traffic densities to validate model robustness:

- **Low Congestion Suite:** Ensures the system does not cycle unnecessarily when lanes are empty.
- **High Congestion Suite:** Validates that "Survivor Bias" deadlocks are cleared and teleportation failures remain below the 15-vehicle threshold.
- **Asymmetric Suite:** Tests system fairness when one axis processes three times the volume of the opposing axis.

10.16 Maintainable Phase

10.16.1 CI / CD

A Continuous Integration (CI) pipeline will be established using GitHub Actions. Upon every push or pull request to the main repository, an automated Ubuntu environment will initialize, install necessary dependencies via pip, and execute the PyTest suite to ensure that new code commits do not break the underlying equilibrium logic.

10.16.2 Deployment Diagram

The deployment architecture consists of a single execution node (the researcher's workstation). Inside this node, two primary runtime environments exist concurrently: the Python 3.x Virtual Environment executing the Game Theory Controller, and the Eclipse SUMO C++ Binary. These two environments communicate locally via a TCP/IP socket strictly established on Port 9000 by the TraCI protocol.

10.16.3 System-Level Test Suites, Test Cases

System-level tests will validate the entire workflow (E2E). A test script will launch the SUMO simulator alongside the controller script, execute a 3600-step (1 hour) simulation, and automatically parse the resulting `tripinfo.xml` to assert that total network throughput exceeds the established fixed-timer baseline of 32,700 vehicles.

10.16.4 SVN or GitHub

Version control is managed via Git, with the central repository hosted on GitHub. Development follows a feature-branch workflow (e.g., branching `feature/predictive-equilibrium` from `main`), allowing experimental iteration phases (like transitioning from reactive to predictive logic) without destabilizing the stable working baseline.

10.16.5 Configuration / Setup and Tool Manual

To run the system, the host machine requires Python 3.7+ and Eclipse SUMO v1.10.0+. The `SUMO_HOME` environment variable must be configured to point to the local installation path. A `requirements.txt` file is provided to install all necessary Python dependencies (such as `traci` and `numpy`). Once configured, users simply execute `python controller.py` from the terminal, which automatically launches the SUMO GUI and begins optimization.